Exemplo de derivação sintática: for em JavaScript

Linguagem escolhida: JavaScript
Fonte: https://tc39.es/ecma262/
Notação: variante da BNF mais especificamente adaptada no padrão ECMAScript (ECMA-262).

1. Código a ser derivado

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

2. Regras gramaticais utilizadas

```
ForStatement :
    for ( VariableDeclarationList ; Expression opt ;
Expression opt ) Statement

Statement :
    Block

Block :
    { StatementList }

StatementList :
    ExpressionStatement

ExpressionStatement :
    Expression ;

Expression :
    UpdateExpression
    RelationalExpression

UpdateExpression :
```

```
    IdentifierReference ++

RelationalExpression :
    IdentifierReference <= NumericLiteral

CallExpression :
    MemberExpression ( Arguments )

MemberExpression :
    console . log

Arguments :
    IdentifierReference

VariableDeclarationList :
    let IdentifierReference = NumericLiteral

IdentifierReference :
    i

NumericLiteral :
    1
    5
```

3. Derivação

```
ForStatement
⇒ for ( VariableDeclarationList ; Expression ; Expression
) Statement
⇒ for ( let IdentifierReference = NumericLiteral ;
RelationalExpression ; UpdateExpression ) Statement
⇒ for ( let i = NumericLiteral ; RelationalExpression ;
UpdateExpression ) Statement
⇒ for ( let i = 1 ; RelationalExpression ;
UpdateExpression ) Statement
⇒ for ( let i = 1 ; IdentifierReference <= NumericLiteral
; UpdateExpression ) Statement
```

```
⇒ for ( let i = 1 ; i <= NumericLiteral ;
UpdateExpression ) Statement
⇒ for ( let i = 1 ; i <= 5 ; UpdateExpression ) Statement
⇒ for ( let i = 1 ; i <= 5 ; IdentifierReference++ )
Statement
⇒ for ( let i = 1 ; i <= 5 ; i++ ) Statement
⇒ for ( let i = 1 ; i <= 5 ; i++ ) Block
⇒ for ( let i = 1 ; i <= 5 ; i++ ) { StatementList }
⇒ for ( let i = 1 ; i <= 5 ; i++ ) { ExpressionStatement
}
⇒ for ( let i = 1 ; i <= 5 ; i++ ) { CallExpression ; }
⇒ for ( let i = 1 ; i <= 5 ; i++ ) { MemberExpression (
Arguments ) ; }
⇒ for ( let i = 1 ; i <= 5 ; i++ ) { console.log (
Arguments ) ; }
⇒ for ( let i = 1 ; i <= 5 ; i++ ) { console.log (
IdentifierReference ) ; }
⇒ for ( let i = 1 ; i <= 5 ; i++ ) { console.log ( i ) ;
}
```

4. Breve explicação textual

O comando for do JavaScript é uma estrutura de repetição controlada por três partes: inicialização (declara e dá valor inicial à variável de controle), condição (avaliada antes de cada repetição — o laço continua enquanto ela for verdadeira) e atualização (executada ao final de cada repetição). Neste exemplo, a variável i começa em 1, o laço repete enquanto i <= 5, incrementando i a cada volta com i++. Dentro do bloco { }, o comando console.log(i) imprime o valor atual de i. A derivação mostra que essa sequência de código não surge de forma arbitrária: ela é válida porque pode ser construída passo a passo a partir das regras da gramática oficial da linguagem, substituindo um não terminal por vez até restar apenas símbolos terminais.

5. Símbolos não terminais e terminais

Não terminais: ForStatement, Statement, Block, StatementList, ExpressionStatement,
Expression, UpdateExpression, RelationalExpression, CallExpression, MemberExpression,

Arguments, VariableDeclarationList, IdentifierReference, NumericLiteral

Terminais: for, (,), ;, let, =, <=, ++, {, }, console, ., log, i, 1, 5